Lab5 report

The main object of this lab is to build a piplined CPU with harzard detection and forwarding component. The CPU can be broken into five stages. Thus, the compilation order is:

Firstly the basic components: AND2.vhd, add.vhd, OR2.vhd, MUX5.vhd, MUX64.vhd, MUX64_3.vhd.

And then the critical components of the CPU: alu.vhd, alucontrol.vhd, cpucontrol.vhd, PC.vhd, registers_p1.vhd, ShiftLeft2.vhd, SignExtend.vhd, lab5_dmem.chd, lab5_imem.vhd.

After that, we should compile the caches between the stages: IFID.vhd, IDEX.vhd, EXMEM.vhd, MEMWB.vhd.

Finally, the top-level design of the entire system: piplinedcpu1.vhd and my own testbench piplinedcpu1_tb.vhd (could be removed if you don't want it).

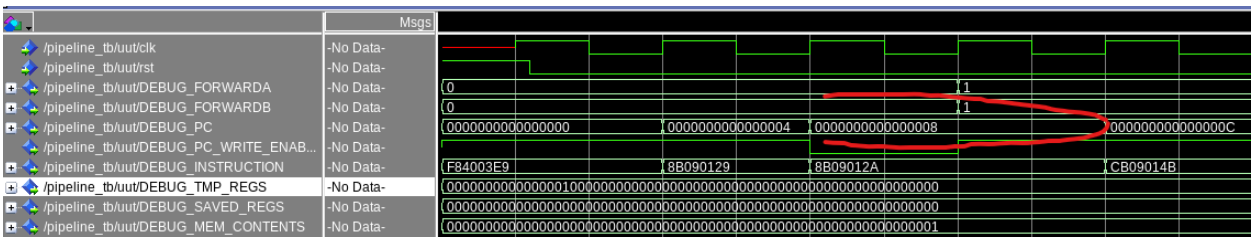
1.Instructions analysis

Using given instructions, we can draw the different stages of operating instructions:

CLK	1	2	3	4	5	6	7	8	9	10	11
LDUR	IF	ID	EX	MEM	WB						
ADD		IF	ID*	ID	EX	MEM	WB				
ADD			IF*	IF	ID	EX	MEM	WB			
SUB					IF	ID	EX	MEM	WB		
STUR						IF	ID	EX	MEM	WB	
STUR							IF	ID	EX	MEM	WB

2.Hazard_detect and forwarding

From the instruction analysis above, we can see that there is only one stall at the third clock cycle, which is also given out as the following figure:

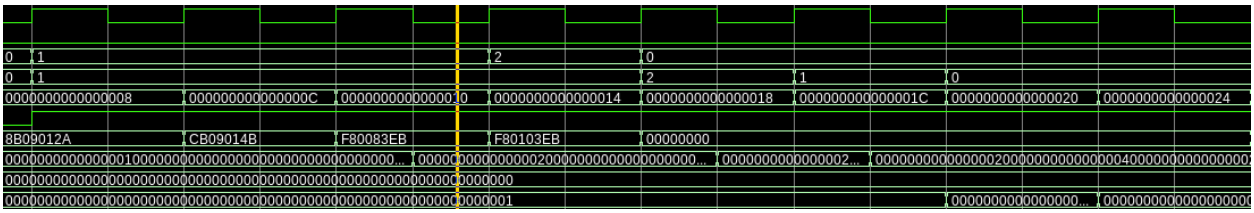


And the DEBUG_PC signal shows how the CPU stalled at the third clock cycle.

According to the chart above, there is several forwardings between:

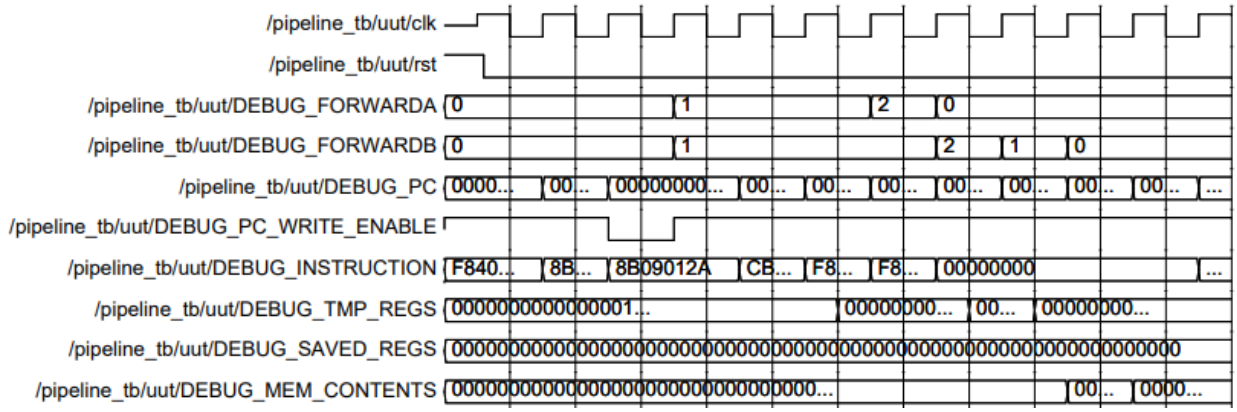
1. LDUR X9, [XZR, 0] and ADD X9, X9, X9;
2. ADD X9, X9, X9 and ADD X10, X9, X9;
3. ADD X10, X9, X9 and SUB X11, X10, X9;
4. SUB X11, X10, X9 and STUR X11, [XZR, 8];
5. SUB X11, X10, X9 and STUR X11, [XZR, 16];

And the Forwarding signals given out at those cycles is:



3.Description of the entire process

The DEBUG signal in the useful range is printed out as follows:

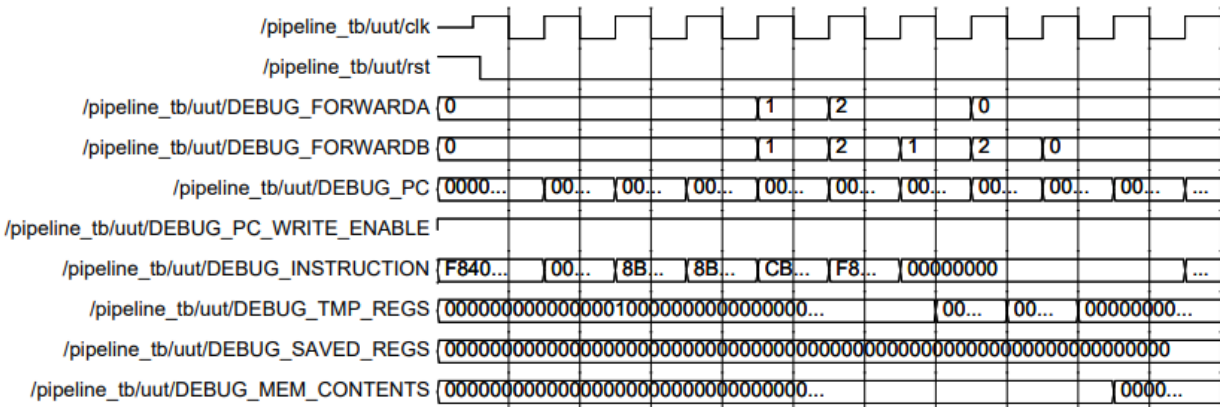


We can see that at the seventh stage the registers value begins to change, which can correspond to the first “ADD” instruction.

The first change of memory contents is at the ninth stage. That is also matched with the table of how instructions operating.

4.Extra Credit

I’ve added a NOP instruction right after the first instruction. And the signal gives out as:



The only difference between NOP-added and non-NOP-added is the output of forwarding. This may due to the original forwarding signal doesn't display well I think.

- There is only one hazard before we add and NOP instruction to the instruction memory. Thus, with a NOP added, we will not have any stall during the process.
- The forwarding values seems change on the given signal. However, it doesn't change the result of our program, including the registers value and dmem value.
- The number of cycle to end the program doesn't change, which is still 11 clock cycles.
- The value of PC when sw occurs is "20" in hex.
- The final state of DMEM and Registers stay still the same: X9=2, X10=4, X11=2 and DMEM(0x8)=0x00000002, DMEM(0x16)=0X00000002.